CSE222 ASSIGNMENT 7-8 REPORT

Sorting Algorithms, Adjacency Matrix, and Graph Coloring

Veysel Cemaloğlu 200104004107

1. INTRODUCTION

This project implements a comprehensive data structures and algorithms system consisting of three main components:

Core Implementation:

- **Sorting Algorithms**: Implemented GTUInsertSort (insertion sort) and GTUQuickSort (quicksort with random pivot) extending the GTUSorter abstract class
- **Graph Data Structure**: Developed MatrixGraph class using adjacency matrix representation with AdjacencyVect for efficient neighbor storage
- **Integration Testing**: Connected all components through a greedy graph coloring algorithm that demonstrates real-world application

Bonus Features Implemented:

- **GTUSelectSort**: Selection sort algorithm implementation (+10 points)
- **Complete AdjacencyVect**: Full Collection interface implementation with all optional methods (+10 points)

The system processes graph input files, sorts vertex degree information, and produces graph coloring solutions using different sorting strategies.

2. ENVIRONMENT

Development Environment:

- **Operating System**: Ubuntu 22.04
- **Java Version**: OpenJDK 11.0.26
- **IDE**: VSCode
- **Build System**: Make (using provided makefile)

Testing Environment:

- Local development machine for initial testing
- Command-line compilation and execution following assignment specifications

3. COMPLEXITY ANALYSIS

Sorting Algorithms:

- **GTUInsertSort:**
  - Time: $O(n^2)$ worst/average case, O(n) best case
  - Space: O(1) - in-place sorting
  - Efficient for small arrays and nearly sorted data
- **GTUQuickSort:**
  - Time: O(n log n) average case, $O(n^2)$ worst case
  - Space: O(log n) due to recursion stack
  - Random pivot selection improves average performance
  - Hybrid approach with partition limit optimizes small subarrays
- **GTUSelectSort:**
  - Time: $O(n^2)$ all cases - always performs same number of comparisons
  - Space: O(1) - in-place sorting

Graph Operations:

- **MatrixGraph:**
  - Space: $O(V^2)$ for adjacency matrix storage
  - setEdge/getEdge: O(1) - direct array Access
  - getNeighbors: O(1) - returns reference to AdjacencyVect
- **AdjacencyVect:**
  - Space: O(V) per vertex
  - add/remove/contains: O(1) - boolean array operations
  - Iterator: O(V) for complete traversal, but only visits true value

Graph Coloring Algorithm:

- Overall complexity: $O(V^2 + E + V \log V)$ where V is vertices, E is edges
- Dominated by sorting vertices by degree and checking color conflicts

4. TESTING

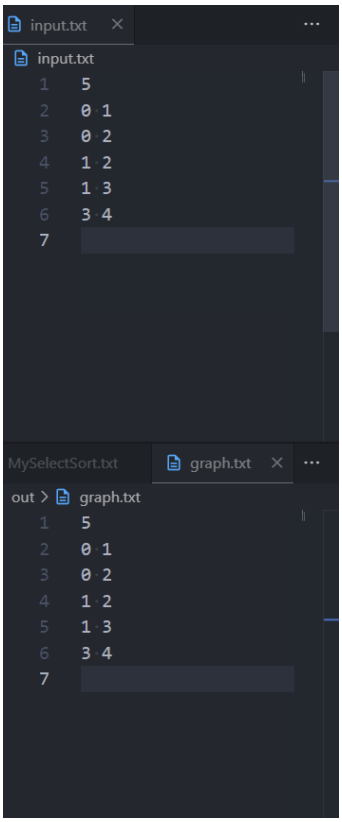
Testing Strategy:

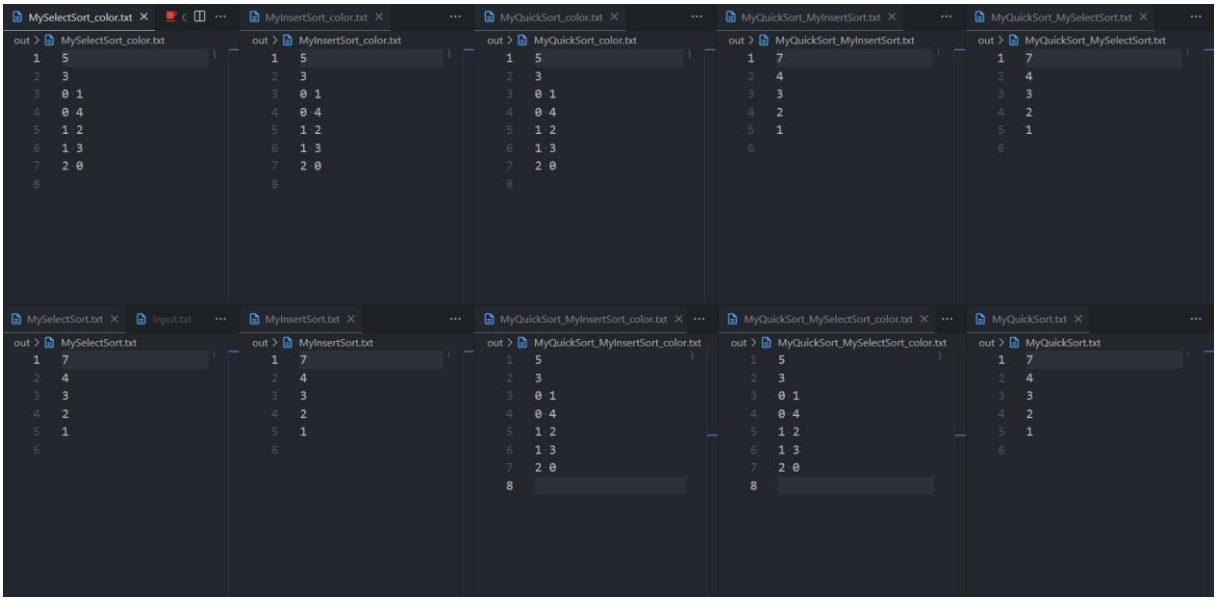
1. **Unit Testing:** Tested each component individually
   - Sorting algorithms with various input sizes and patterns
   - Graph operations with different graph structures
   - AdjacencyVect collection operations
2. **Integration Testing:**
   - Used provided Main.java to test complete workflow
   - Verified file I/O operations with sample graph files
   - Tested graph coloring algorithm with different sorting strategies
3. **Edge Case Testing:**
   - Already sorted and reverse-sorted arrays
   - Dense and sparse graphs
4. **Performance Testing:**
   - Compared sorting algorithm performance on different input sizes

Test Data:

Small graphs (5-10 vertices) for correctness verification

Various graph structures: complete, sparse, tree-like, and random


input.txt
5
0 1
0 2
1 2
1 3
3 4
MySelectSort.txt
graph.txt
out > graph.txt
5
0 1
0 2
1 2
1 3
3 4


MySelectSort_color.txt
5
3
0 1
0 4
1 2
1 3
2 0
MyInsertSort_color.txt
5
3
0 1
0 4
1 2
1 3
2 0
MyQuickSort_color.txt
5
3
0 1
0 4
1 2
1 3
2 0
MyQuickSort_MyInsertSort.txt
7
4
3
2
1
MyQuickSort_MySelectSort.txt
7
4
3
2
1
MySelectSort.txt
input.txt
7
4
3
2
1
MyInsertSort.txt
7
4
3
2
1
MyQuickSort_MyInsertSort_color.txt
5
3
0 1
0 4
1 2
1 3
2 0
MyQuickSort_MySelectSort_color.txt
5
3
0 1
0 4
1 2
1 3
2 0
MyQuickSort.txt
7
4
3
2
1

5. AI USAGE

AI Assistance Areas:

- **Code Structure Guidance:** Used AI to understand Java Collection interface requirements and best practices for implementing custom iterators
- **Algorithm Optimization:** Consulted AI for QuickSort partition strategies and hybrid sorting approaches
- **JavaDoc Documentation:** AI helped format comprehensive method documentation

Implementation Approach:

- AI provided conceptual guidance and code structure suggestions
- AI-generated code was reviewed, modified, and tested thoroughly
- Used AI as a reference tool rather than for direct code generation

6. CHALLENGES

Technical Challenges:

- **Collection Interface Implementation:** Understanding the nuances of Java's Collection interface, especially implementing a custom iterator that only visits "true" values in AdjacencyVect
- **Generic Type Handling:** Properly implementing generic sorting methods with type safety and comparator usage
- **Undirected Graph Management:** Ensuring edge consistency in MatrixGraph by maintaining symmetric adjacency relationships
- **Memory Efficiency:** Balancing space complexity in adjacency matrix representation while maintaining performance

Design Challenges:

- **Integration Complexity:** Ensuring all components work seamlessly with the provided graph coloring algorithm
- **File I/O Format:** Matching exact input/output specifications for graph representation and coloring solutions
- **Error Handling:** Implementing robust bounds checking and null pointer prevention across all components

Solutions Applied:

- Code review and refactoring for clarity and efficiency
- Documentation-driven development to understand requirements clearly

7. CONCLUSION

Key Learning Outcomes:

- **Algorithm Implementation:** Gained deep understanding of sorting algorithm internals and their practical trade-offs. QuickSort's hybrid approach with insertion sort for small partitions demonstrates real-world optimization techniques.
- **Data Structure Design:** Learned to implement complex data structures (adjacency matrix) using simpler components (boolean arrays) while maintaining clean interfaces through Java's Collection framework.
- **Software Integration:** Experienced how individual components must work together in larger systems, emphasizing the importance of interface design and specification compliance.
- **Performance Analysis:** Developed skills in analyzing time/space complexity and making informed decisions about algorithm selection based on problem constraints.

- **Object-Oriented Design:** Applied inheritance, polymorphism, and generic programming effectively to create reusable and maintainable code.

Project Impact:

This assignment successfully demonstrates the practical application of fundamental algorithms and data structures in solving real-world problems. The graph coloring application showcases how sorting algorithms directly impact optimization problem solutions, reinforcing the interconnected nature of algorithmic thinking.

The implementation achieves all core requirements plus bonus objectives, providing a solid foundation for understanding advanced graph algorithms and optimization techniques in computer science.